PROJECT NOTES · BACKEND

EduSystem Backend

Laravel API, step by step

A rebuild guide to the EduSystem Laravel API that serves the React client. It covers Sanctum bearer tokens and their lifecycle, enums and models, Form Requests, policies and API Resources, an enrollment service that cannot oversell a course, every module from categories to dashboards, delete rules that protect your data, and a 25-test suite. Each step shows clean code that passes the test suite, and flags every bug or security gap fixed against the repo.

Laravel 13.35 · PHP 8.4 · Sanctum 4.3 · SQLite · PHPUnit 12 · Pint · Form Requests · Policies · API Resources

Part A · Overview
Architecture, stack, folders, role matrix

Part B · Backend
Models, validation, auth, policies, every module

Part C · Database
Schema, delete rules, indexes, seed data

Part D · Security
Threat model, OWASP mapping, prod checklist

Part E · Testing
PHPUnit feature tests, 25 passing

Part F · Reference
Env, commands, version notes, bugs fixed

Source: education-system/backend · companion to “EduSystem Frontend” notes

Sommaire

Part A · Overview 3
01 What it is & how the pieces fit 3
02 Tech stack & folder structure 4
03 Roles & what each one can do 4

Part B · Backend 6
04 Setup: install, environment, SQLite 6
05 Enums: one source of truth for roles, levels & statuses 7
06 Models: User & Role 7
07 Models: Course, Category, Enrollment & unique slugs 9
08 Bootstrap: middleware, strict models & one error shape 12
09 Validation with Form Requests 13
10 API Resources: shape the JSON, hide private fields 15
11 Authentication: register & login with Sanctum tokens 17
12 Token lifecycle: me, profile, logout, expiry & pruning 18
13 Authorization: role middleware + policies 19
14 Categories module 21
15 Courses module 22
16 Enrollment service: seats, re-enroll, consistent status 24
17 Enrollments controller 25
18 Users module (admin) 26
19 Dashboard: one endpoint, three shapes 27
20 Routes 29
21 Try it with curl 30
22 Backend lifecycle cheat table 31

Part C · Database 32
23 Schema & relations 32
24 Migrations: delete rules & indexes 32
25 Seeding & demo accounts 34

Part D · Security 35
26 Threat model & OWASP mapping 35
27 Production hardening checklist 36

Part E · Testing 37
28 Test setup & helpers 37
29 Feature tests: auth, catalog, enrollments 38

Part F · Reference 40
30 Environment variables & commands 40
31 Version notes & upgrade path 41
32 Bugs & gaps fixed vs. the repo 41

PART A

Overview

What the API does, how a request flows, the stack, and who may do what.

STEP 01 What it is & how the pieces fit

EduSystem's backend is a stateless JSON API in Laravel 13. The React SPA calls it with a **Sanctum bearer token**. Admins run the catalog and users, students enroll, and parents follow their children.

```
React SPA (localhost:3000)                Laravel API (localhost:8000/api)
──────────────────────────                ──────────────────────────────────────────
axios + Authorization: Bearer <token> ──► HandleCors → UseSanctumGuard → throttle:api
                                            │
                                            ├─ auth:sanctum   (401 if token missing/expired)
                                            ├─ role:admin     (403 for admin-only groups)
                                            ▼
                                          FormRequest  (422 + errors{} if invalid)
                                            ▼
                                          Controller ── Gate::authorize → Policy (403/404)
                                            │            Service (EnrollmentService, 409/422)
                                            ▼
                                          Eloquent models ──► SQLite (database/database.sqlite)
                                            ▼
                                          API Resource  ──► JSON (no wrapping, no private fields)
```

Layer	Where	Answers the question
Route + middleware	`routes/api.php`, `bootstrap/app.php`	Who may reach an endpoint at all
Form Request	`app/Http/Requests`	Shape and validity of the input
Policy	`app/Policies`	May *this* user touch *this* record?
Service	`app/Services`	Business rules that span models (seats, re-enroll)
Model	`app/Models`	Relations, casts, scopes, slugs
Resource	`app/Http/Resources`	Exactly which fields leave the server

WHY

Every layer has one job. A controller method is 3–8 lines: authorize, call the model or service, return a resource.

STEP 02 Tech stack & folder structure

This is the stock Laravel 13 skeleton plus Sanctum. There is no Blade UI: the only web route is the welcome page.

Tool	Version	Notes
PHP	8.4 (8.3+ required by Laravel 13)	
laravel/framework	13.35	slim skeleton: `bootstrap/app.php`, no HTTP Kernel
laravel/sanctum	4.3	API tokens in `personal_access_tokens`
Database	SQLite	file at `database/database.sqlite`; FK constraints on
phpunit	12.5	in-memory SQLite in `phpunit.xml`
laravel/pint	1.27	code style (`vendor/bin/pint`)

```
backend/
├── app/
│   ├── Enums/                RoleName, CourseLevel, EnrollmentStatus
│   ├── Exceptions/           BusinessRuleException (422 / 409)
│   ├── Http/
│   │   ├── Controllers/Api/  Auth, Course, CourseCategory, Enrollment, User, Dashboard
│   │   ├── Middleware/       EnsureRole, UseSanctumGuard
│   │   ├── Requests/         one FormRequest per write (+ Auth/)
│   │   └── Resources/        User, Course, Category, Enrollment
│   ├── Models/               User, Role, Course, CourseCategory, Enrollment (+ Concerns/HasUniqueSlug)
│   ├── Policies/             Course, CourseCategory, Enrollment, User
│   ├── Providers/            AppServiceProvider (rate limiters, strict models)
│   ├── Rules/                UserWithRole
│   └── Services/             EnrollmentService
├── bootstrap/app.php         routing, middleware, exception rendering
├── config/                   cors.php, sanctum.php
├── database/                 migrations/, seeders/, database.sqlite
├── routes/                   api.php, console.php (scheduler), web.php
└── tests/Feature/            AuthTest, CatalogTest, EnrollmentTest
```

STEP 03 Roles & what each one can do

There are three roles, stored in the `roles` table and mirrored by the `RoleName` enum. Parents are linked to students through `users.parent_id`.

Action	Guest	Admin	Student	Parent
Browse published courses & active categories	✓	✓	✓	✓
See drafts / inactive categories	—	✓	—	—
Register	student or parent	—	—	—
Create / edit / delete courses & categories	—	✓	—	—
Manage users	—	✓ (not self-delete / self-demote)	own profile	own profile
Enroll	—	any student	self	—

Action	Guest	Admin	Student	Parent
Drop enrollment	—	✓	own	—
Set progress / grade / status	—	✓	—	—
View enrollments	—	all	own	children's
Dashboard	—	system stats	own stats	per-child stats

> **NOTE**
> Admins are also the instructors: `courses.instructor_id` must point to an admin account.

PART B

Backend

Models, validation, auth, authorization and one step per module, built in the order you'd write them.

STEP 04 Setup: install, environment, SQLite

Clone the repo, install, then create and seed the database. Sanctum ships with its migration, and API routes are registered in `bootstrap/app.php`.

terminal

```
cd education-system/backend
composer install
cp .env.example .env && php artisan key:generate
touch database/database.sqlite          # PowerShell: ni database/database.sqlite
php artisan migrate:fresh --seed         # roles, 4 demo users, 6 categories, 9 courses
php artisan serve                        # http://127.0.0.1:8000
```

.env

```
APP_URL=http://localhost:8000
APP_DEBUG=true                    # false in production: debug pages leak stack traces
DB_CONNECTION=sqlite              # DB_DATABASE defaults to database/database.sqlite

CORS_ALLOWED_ORIGINS=http://localhost:3000,http://127.0.0.1:3000
SANCTUM_EXPIRATION=10080          # token lifetime in minutes (7 days)
```

config/cors.php

```
'paths' => ['api/*', 'sanctum/csrf-cookie'],

    'allowed_methods' => ['*'],

    'allowed_origins' => explode(',', env('CORS_ALLOWED_ORIGINS', 'http://localhost:3000,http://127.0.0.1:3000')),

    'allowed_origins_patterns' => [],

    'allowed_headers' => ['*'],

    'exposed_headers' => [],
```

NOTE

The SPA sends `Authorization: Bearer …`, not cookies. You don't need `supports_credentials` or `SANCTUM_STATEFUL_DOMAINS` for this flow, but they do no harm.

STEP 05 Enums: one source of truth for roles, levels & statuses

Backed enums replace magic strings. Models cast to them, validation uses `Rule::enum()`, and JSON still shows the plain value.

app/Enums/RoleName.php

```php
namespace App\Enums;

enum RoleName: string
{
    case Admin = 'admin';
    case Student = 'student';
    case Parent = 'parent';

    /** Roles a visitor may pick on the public register form. */
    public static function selfRegistrable(): array
    {
        return [self::Student, self::Parent];
    }
}
```

app/Enums/EnrollmentStatus.php

```php
namespace App\Enums;

enum EnrollmentStatus: string
{
    case Active = 'active';
    case Completed = 'completed';
    case Dropped = 'dropped';
}
```

> **NOTE**
> `CourseLevel` is the same shape: `beginner | intermediate | advanced`.

> **VS. THE REPO**
> The repo compared strings like `'admin'` and `'dropped'` in a dozen places. A typo there fails silently; an enum case fails at once.

STEP 06 Models: User & Role

Laravel 13 declares `#[Fillable]` and `#[Hidden]` as class attributes. `$with = ['role']` loads the role together with the user, because nearly every permission check reads it.

app/Models/User.php

```php
namespace App\Models;

// use … (imports omitted)

#[Fillable(['name', 'email', 'password', 'role_id', 'parent_id', 'phone', 'address', 'date_of_birth'])]
#[Hidden(['password', 'remember_token'])]
class User extends Authenticatable
{
    /** @use HasFactory<UserFactory> */
    use HasApiTokens, HasFactory, Notifiable;

    // Every permission check needs the role: load it with the user, once.
```

```
    protected $with = ['role'];

    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
            'date_of_birth' => 'date',
        ];
    }

    public function role(): BelongsTo
    {
        return $this->belongsTo(Role::class);
    }

    public function parent(): BelongsTo
    {
        return $this->belongsTo(User::class, 'parent_id');
    }

    public function children(): HasMany
    {
        return $this->hasMany(User::class, 'parent_id');
    }

    public function enrollments(): HasMany
    {
        return $this->hasMany(Enrollment::class, 'student_id');
    }

    public function taughtCourses(): HasMany
    {
        return $this->hasMany(Course::class, 'instructor_id');
    }

    public function hasRole(RoleName ...$roles): bool
    {
        return in_array($this->role?->name, $roles, true);
    }

    public function isAdmin(): bool
    {
        return $this->hasRole(RoleName::Admin);
    }

    public function isStudent(): bool
    {
        return $this->hasRole(RoleName::Student);
    }

    public function isParent(): bool
    {
        return $this->hasRole(RoleName::Parent);
    }

    public function isParentOf(int $studentId): bool
    {
        return $this->isParent() && $this->children()->whereKey($studentId)->exists();
    }

    #[Scope]
    protected function withRole(Builder $query, RoleName $role): void
    {
        $query->whereHas('role', fn (Builder $q) => $q->where('name', $role));
```

```
    }
}
```

app/Models/Role.php

```
#[Fillable(['name', 'description'])]
class Role extends Model
{
    protected function casts(): array
    {
        return ['name' => RoleName::class];
    }

    public function users(): HasMany
    {
        return $this->hasMany(User::class);
    }

    public static function idFor(RoleName $name): ?int
    {
        return static::where('name', $name)->value('id');
    }
}
```

> **TIP**
> `'password' => 'hashed'` hashes on assignment, so controllers never call `Hash::make()`, and the cast never double-hashes an existing hash.

STEP 07 Models: Course, Category, Enrollment & unique slugs

Slugs used to be generated by three copies of the same `while` loop across two controllers. They now live in one trait that runs on `saving`.

app/Models/Concerns/HasUniqueSlug.php

```
namespace App\Models\Concerns;

use Illuminate\Support\Str;

/**
 * Fills `slug` from slugSource() on create, and again when the source changes.
 */
trait HasUniqueSlug
{
    abstract protected function slugSource(): string;

    protected static function bootHasUniqueSlug(): void
    {
        static::saving(function (self $model) {
            $source = $model->slugSource();

            if ($model->exists && ! $model->isDirty($source)) {
                return;
            }

            $model->slug = $model->uniqueSlug((string) $model->{$source});
        });
    }

    private function uniqueSlug(string $value): string
    {
```

```
        $base = Str::slug($value) ?: 'item';
        $slug = $base;
        $i = 1;

        while (static::where('slug', $slug)
            ->when($this->exists, fn ($q) => $q->whereKeyNot($this->getKey()))
            ->exists()) {
            $slug = $base.'-'.$i++;
        }

        return $slug;
    }
}
```

app/Models/Course.php

```
#[Fillable([
    'title', 'description', 'category_id', 'instructor_id', 'level',
    'price', 'duration_hours', 'image', 'is_published', 'max_students',
])]
class Course extends Model
{
    use HasFactory, HasUniqueSlug;

    protected function casts(): array
    {
        return [
            'level' => CourseLevel::class,
            'price' => 'decimal:2',
            'is_published' => 'boolean',
        ];
    }

    protected function slugSource(): string
    {
        return 'title';
    }

    public function category(): BelongsTo
    {
        return $this->belongsTo(CourseCategory::class, 'category_id');
    }

    public function instructor(): BelongsTo
    {
        return $this->belongsTo(User::class, 'instructor_id');
    }

    public function enrollments(): HasMany
    {
        return $this->hasMany(Enrollment::class);
    }

    #[Scope]
    protected function published(Builder $query): void
    {
        $query->where('is_published', true);
    }

    /** Filters coming from GET /courses (already validated). */
    #[Scope]
    protected function filter(Builder $query, array $filters): void
    {
        $query
            ->when($filters['category_id'] ?? null, fn ($q, $id) => $q->where('category_id', $id))
            ->when($filters['level'] ?? null, fn ($q, $level) => $q->where('level', $level))
```

```
            ->when($filters['search'] ?? null, fn ($q, $term) => $q->where(fn ($q) => $q
                ->where('title', 'like', "%{$term}%")
                ->orWhere('description', 'like', "%{$term}%")));
    }

    #[Scope]
    protected function withActiveCount(Builder $query): void
    {
        $query->withCount(['enrollments' => fn ($q) => $q->where('status', EnrollmentStatus::Active)]);
    }

    public function isFull(): bool
    {
        return $this->max_students !== null
            && $this->enrollments()->where('status', EnrollmentStatus::Active)->count() >= $this->max_students;
    }
}
```

app/Models/Enrollment.php

```
#[Fillable(['student_id', 'course_id', 'status', 'progress', 'grade', 'enrolled_at', 'completed_at'])]
class Enrollment extends Model
{
    use HasFactory;

    protected function casts(): array
    {
        return [
            'status' => EnrollmentStatus::class,
            'progress' => 'integer',
            'grade' => 'decimal:2',
            'enrolled_at' => 'datetime',
            'completed_at' => 'datetime',
        ];
    }

    public function student(): BelongsTo
    {
        return $this->belongsTo(User::class, 'student_id');
    }

    public function course(): BelongsTo
    {
        return $this->belongsTo(Course::class);
    }
}
```

WHY

`slug` is not fillable. Clients can't set it, and with strict mode (Step 08) a stray `slug` key throws an error in dev instead of being ignored silently.

NOTE

`#[Scope]` on a protected method (Laravel 12+) replaces the old `scopePublished()` naming convention. You still call it as `Course::published()`.

STEP 08 Bootstrap: middleware, strict models & one error shape

Every API error is `{ "message": "…" }`. Validation adds `errors{}`. The frontend reads one shape everywhere.

bootstrap/app.php

```
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->api(prepend: [UseSanctumGuard::class]);
        $middleware->throttleApi();
        $middleware->alias(['role' => EnsureRole::class]);
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        $exceptions->shouldRenderJsonWhen(
            fn (Request $request) => $request->is('api/*') || $request->expectsJson(),
        );

        $exceptions->render(fn (ModelNotFoundException|NotFoundHttpException $e, Request $request) => $request-
>is('api/*')
            ? response()->json(['message' => 'Resource not found.'], 404)
            : null);

        $exceptions->render(fn (AuthenticationException $e, Request $request) => $request->is('api/*')
            ? response()->json(['message' => 'Unauthenticated. Please log in again.'], 401)
            : null);
    })->create();
```

app/Http/Middleware/UseSanctumGuard.php

```
/**
 * Makes `sanctum` the default guard on every API route, so $request->user()
 * and policies see the bearer-token user even on public routes like GET /courses.
 */
class UseSanctumGuard
{
    public function handle(Request $request, Closure $next): Response
    {
        Auth::shouldUse('sanctum');

        return $next($request);
    }
}
```

app/Exceptions/BusinessRuleException.php

```
/**
 * A request that is valid but breaks a domain rule ("course is full").
 * Laravel calls render() itself, so services can just throw.
 */
class BusinessRuleException extends RuntimeException
{
    public function __construct(string $message, private int $status = 422)
    {
        parent::__construct($message);
    }

    public static function conflict(string $message): self
    {
```

```
        return new self($message, 409);
    }

    public function render(): JsonResponse
    {
        return response()->json(['message' => $this->getMessage()], $this->status);
    }
}
```

app/Providers/AppServiceProvider.php

```
    public function boot(): void
    {
        // Fail loudly in dev/tests on N+1 lazy loads and unfillable attributes.
        Model::shouldBeStrict(! $this->app->isProduction());

        // The React client expects plain objects/arrays, not { data: ... }.
        JsonResource::withoutWrapping();

        RateLimiter::for('api', fn (Request $request) => Limit::perMinute(120)
            ->by($request->user()?->id ?: $request->ip()));

        // Brute-force guard for login/register: per e-mail + IP.
        RateLimiter::for('auth', fn (Request $request) => Limit::perMinute(5)
            ->by(Str::lower((string) $request->input('email')).'|'.$request->ip()));
    }
}
```

Status	Meaning	Raised by
401	no / invalid / expired token	`auth:sanctum`
403	authenticated but not allowed	`role:` middleware, policies
404	missing record, or a draft a guest may not see	route binding, `denyAsNotFound()`
409	would orphan data (category with courses)	`BusinessRuleException::conflict()`
422	invalid input, or a broken rule (course full)	FormRequest, `BusinessRuleException`
429	too many attempts	`throttle:auth` / `throttle:api`

VS. THE REPO

The repo resolved optional users with `$request->user('sanctum')` inside each controller. Making `sanctum` the default guard for the API group lets policies and `$request->user()` work the same on public and private routes.

STEP 09 Validation with Form Requests

Validation moves out of controllers. One request class serves both `POST` (fields required) and `PUT` (fields `sometimes`).

app/Rules/UserWithRole.php

```
/** 'parent_id' => ['nullable', UserWithRole::of(RoleName::Parent)] */
final class UserWithRole
{
    public static function of(RoleName $role): Exists
    {
        return Rule::exists('users', 'id')->where('role_id', Role::idFor($role));
    }
}
```

app/Http/Requests/Auth/RegisterRequest.php

```
class RegisterRequest extends FormRequest
{
    public function rules(): array
    {
        return [
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'email', 'max:255', 'unique:users'],
            'password' => ['required', 'confirmed', Password::min(8)],
            'role' => ['required', Rule::enum(RoleName::class)->only(RoleName::selfRegistrable())],
            'phone' => ['nullable', 'string', 'max:20'],
            'address' => ['nullable', 'string', 'max:500'],
            'date_of_birth' => ['nullable', 'date', 'before:today'],
            'parent_id' => ['nullable', 'prohibited_unless:role,student', UserWithRole::of(RoleName::Parent)],
        ];
    }
}
```

app/Http/Requests/UserRequest.php

```
/**
 * POST /users (admin) and PUT /users/{user} (admin, or the user on their own account).
 * Fields a non-admin may not touch are dropped in validated().
 */
class UserRequest extends FormRequest
{
    private const ADMIN_ONLY = ['email', 'password', 'role_id', 'parent_id'];

    public function rules(): array
    {
        $creating = $this->isMethod('post');
        $required = $creating ? 'required' : 'sometimes';
        $user = $this->route('user');

        return [
            'name' => [$required, 'string', 'max:255'],
            'email' => [$required, 'string', 'email', 'max:255', Rule::unique('users')->ignore($user)],
            'password' => [$creating ? 'required' : 'nullable', Password::min(8)],
            'role_id' => [$required, 'integer', 'exists:roles,id'],
            'phone' => ['nullable', 'string', 'max:20'],
            'address' => ['nullable', 'string', 'max:500'],
            'date_of_birth' => ['nullable', 'date', 'before:today'],
            'parent_id' => [
                'nullable',
                'prohibited_unless:role_id,'.Role::idFor(RoleName::Student),
                UserWithRole::of(RoleName::Parent),
            ],
        ];
    }

    public function validated($key = null, $default = null): mixed
    {
        $data = parent::validated();

        if (! $this->user()->isAdmin()) {
            $data = array_diff_key($data, array_flip(self::ADMIN_ONLY));
        }

        if (empty($data['password'])) {
            unset($data['password']);
        }

        return data_get($data, $key, $default);
```

```
    }
}
```

app/Http/Requests/StoreEnrollmentRequest.php

```
class StoreEnrollmentRequest extends FormRequest
{
    public function rules(): array
    {
        $isAdmin = $this->user()->isAdmin();

        return [
            'course_id' => ['required', 'integer', 'exists:courses,id'],
            // Admins enroll a student; students always enroll themselves.
            'student_id' => $isAdmin
                ? ['required', 'integer', UserWithRole::of(RoleName::Student)]
                : ['nullable'],
        ];
    }

    public function studentId(): int
    {
        return $this->user()->isAdmin()
            ? (int) $this->validated('student_id')
            : $this->user()->id;
    }
}
```

WHY

`UserRequest` drops admin-only fields (email, password, role, parent) for non-admins, so a student editing their own record can't promote themselves.

VS. THE REPO

The repo checked `parent_id` with a bare `exists:users,id` in `POST/PUT /users`, so any account (even an admin) could become a "parent". Admins could also enroll a parent or another admin as a "student".

STEP 10 API Resources: shape the JSON, hide private fields

Resources decide what leaves the server. Fields the React app reads keep their names, so the frontend needs no changes.

app/Http/Resources/UserResource.php

```
/** @mixin User */
class UserResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        // Contact details only for admins, the user, or their parent.
        $private = (bool) $request->user()?->can('view', $this->resource);

        return [
            'id' => $this->id,
            'name' => $this->name,
            'email' => $this->when($private, $this->email),
            'role_id' => $this->role_id,
            'parent_id' => $this->when($private, $this->parent_id),
            'phone' => $this->when($private, $this->phone),
            'address' => $this->when($private, $this->address),
            'date_of_birth' => $this->when($private, fn () => $this->date_of_birth?->toDateString()),
```

```
            'created_at' => $this->created_at,
            'role' => $this->whenLoaded('role', fn () => [
                'id' => $this->role->id,
                'name' => $this->role->name,
            ]),
            'parent' => new UserResource($this->whenLoaded('parent')),
            'children' => UserResource::collection($this->whenLoaded('children')),
            'enrollments' => EnrollmentResource::collection($this->whenLoaded('enrollments')),
        ];
    }
}
```

app/Http/Resources/CourseResource.php

```
/** @mixin Course */
class CourseResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id' => $this->id,
            'title' => $this->title,
            'slug' => $this->slug,
            'description' => $this->description,
            'category_id' => $this->category_id,
            'instructor_id' => $this->instructor_id,
            'level' => $this->level,
            'price' => $this->price,
            'duration_hours' => $this->duration_hours,
            'image' => $this->image,
            'is_published' => $this->is_published,
            'max_students' => $this->max_students,
            'enrollments_count' => $this->whenCounted('enrollments'),
            'created_at' => $this->created_at,
            'category' => new CategoryResource($this->whenLoaded('category')),
            // Public catalog: the instructor's name only, never contact details.
            'instructor' => $this->whenLoaded('instructor', fn () => [
                'id' => $this->instructor->id,
                'name' => $this->instructor->name,
            ]),
            'enrollment' => new EnrollmentResource($this->whenLoaded('enrollment')),
        ];
    }
}
```

VS. THE REPO

The repo returned raw models. `GET /courses` is public and loaded `instructor`, so **any visitor got the instructor's email, phone, address and date of birth**. The public catalog now exposes `{id, name}` only.

NOTE

`JsonResource::withoutWrapping()` (Step 08) keeps responses as plain objects and arrays, not `{ data: … }`, which is what the SPA expects.

STEP 11 Authentication: register & login with Sanctum tokens

Register and login both return `{ user, token }`. The token is a plain-text string shown once; the database stores only its SHA-256 hash.

app/Http/Controllers/Api/AuthController.php

```
class AuthController extends Controller
{
    public function register(RegisterRequest $request): JsonResponse
    {
        $data = $request->validated();

        $user = User::create([
            ...collect($data)->except('role')->all(),
            'role_id' => Role::idFor(RoleName::from($data['role'])),
        ]);

        return $this->tokenResponse($user->load('role'), 201);
    }

    public function login(LoginRequest $request): JsonResponse
    {
        $user = User::where('email', $request->validated('email'))->first();

        if (! $user || ! Hash::check($request->validated('password'), $user->password)) {
            throw ValidationException::withMessages([
                'email' => ['The provided credentials are incorrect.'],
            ]);
        }

        return $this->tokenResponse($user);
    }
```

app/Http/Controllers/Api/AuthController.php

```
    private function tokenResponse(User $user, int $status = 200): JsonResponse
    {
        $token = $user->createToken('api')->plainTextToken;
        Auth::setUser($user); // so UserResource treats the caller as "self"

        return response()->json(['user' => new UserResource($user), 'token' => $token], $status);
    }
}
```

Phase	How	Where
Create	`$user->createToken('api')`	row in `personal_access_tokens` (hashed)
Send	client stores the token, sends `Authorization: Bearer 3|abc…`	SPA: memory / localStorage
Verify	`auth:sanctum` hashes it and looks it up	rejects it if older than `sanctum.expiration`
Revoke	`currentAccessToken()->delete()`	logout, password change

VS. THE REPO

Login and register sit behind `throttle:auth`, 5 attempts a minute per e-mail and IP. The repo had no rate limit, so passwords could be brute-forced without limit.

STEP 12 Token lifecycle: me, profile, logout, expiry & pruning

A token dies three ways: logout, expiry, or a password change. Expired rows are pruned every night.

app/Http/Controllers/Api/AuthController.php

```
public function logout(Request $request): JsonResponse
{
    $request->user()->currentAccessToken()->delete();

    return response()->json(['message' => 'Logged out successfully.']);
}

public function me(Request $request): UserResource
{
    return new UserResource($request->user()->load(['parent', 'children']));
}

public function updateProfile(UpdateProfileRequest $request): UserResource
{
    $user = $request->user();
    $data = collect($request->validated())->except('current_password')->filter(
        fn ($value, $key) => $key !== 'password' || filled($value)
    )->all();

    $user->update($data);

    if (isset($data['password'])) {
        // New password: sign out every other device.
        $user->tokens()->whereKeyNot($user->currentAccessToken()->getKey())->delete();
    }

    return new UserResource($user);
}
```

app/Http/Requests/Auth/UpdateProfileRequest.php

```
class UpdateProfileRequest extends FormRequest
{
    public function rules(): array
    {
        return [
            'name' => ['sometimes', 'required', 'string', 'max:255'],
            'phone' => ['nullable', 'string', 'max:20'],
            'address' => ['nullable', 'string', 'max:500'],
            'date_of_birth' => ['nullable', 'date', 'before:today'],
            'password' => ['nullable', 'confirmed', Password::min(8)],
            // A stolen token alone must not be enough to take over the account.
            'current_password' => ['required_with:password', 'current_password'],
        ];
    }
}
```

routes/console.php

```
use Illuminate\Support\Facades\Schedule;

// Expired tokens are rejected at once, but their rows stay until pruned.
Schedule::command('sanctum:prune-expired --hours=24')->daily();
```

crontab (server)

```
* * * * * cd /var/www/backend && php artisan schedule:run >> /dev/null 2>&1
```

VS. THE REPO

Changing a password now needs `current_password` and signs out every other device. Before, a stolen token was enough to set a new password and lock the owner out. **Frontend:** add a "Current password" field to `Profile.jsx`.

STEP 13 Authorization: role middleware + policies

Two layers. Middleware closes whole route groups by role; policies decide per record. Controllers call `Gate::authorize()` and never write `abort(403)` by hand.

app/Http/Middleware/EnsureRole.php

```php
/** Usage: ->middleware('role:admin') or 'role:admin,parent'. */
class EnsureRole
{
    public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        $allowed = array_map(RoleName::from(...), $roles);

        if (! $request->user()?->hasRole(...$allowed)) {
            return response()->json(['message' => 'You do not have permission to perform this action.'], 403);
        }

        return $next($request);
    }
}
```

app/Policies/EnrollmentPolicy.php

```php
class EnrollmentPolicy
{
    public function view(User $user, Enrollment $enrollment): bool
    {
        return $user->isAdmin()
            || $enrollment->student_id === $user->id
            || $user->isParentOf($enrollment->student_id);
    }

    public function create(User $user): bool
    {
        return $user->isAdmin() || $user->isStudent();
    }

    /** Progress, grade and status are graded by staff only. */
    public function update(User $user, Enrollment $enrollment): bool
    {
        return $user->isAdmin();
    }

    /** "Delete" = drop. Parents have read-only access. */
    public function delete(User $user, Enrollment $enrollment): bool
    {
        return $user->isAdmin()
            || ($user->isStudent() && $enrollment->student_id === $user->id);
    }
}
```

app/Policies/UserPolicy.php

```php
class UserPolicy
{
    public function view(User $auth, User $user): bool
    {
        return $auth->isAdmin() || $auth->is($user) || $auth->isParentOf($user->id);
    }

    public function update(User $auth, User $user): bool
    {
        return $auth->isAdmin() || $auth->is($user);
    }

    public function changeRole(User $auth, User $user): bool
    {
        return $auth->isAdmin() && ! $auth->is($user);
    }

    public function delete(User $auth, User $user): bool
    {
        return $auth->isAdmin() && ! $auth->is($user);
    }
}
```

app/Policies/CoursePolicy.php

```php
class CoursePolicy
{
    // ?User: guests may call this policy (public catalog).
    // 404, not 403: a draft should look like it doesn't exist.
    public function view(?User $user, Course $course): Response
    {
        return $course->is_published || $user?->isAdmin()
            ? Response::allow()
            : Response::denyAsNotFound();
    }
}
```

Question	Lives in
Admin-only CRUD (courses, categories, user list)	`role:admin` on the route group
"Own record" rules (enrollments, user show/update)	Policy method
Public record a guest may or may not see	Policy with `?User` + `denyAsNotFound()`
Which fields a viewer may edit	FormRequest (`UserRequest::validated()`)
Which fields a viewer may see	Resource (`$this->when($private, …)`)

TIP

Policies are auto-discovered: `App\Models\Course` → `App\Policies\CoursePolicy` . No registration needed.

STEP 14 Categories module

Guests see only active categories with published-course counts; admins see everything. A category that still has courses can't be deleted.

app/Http/Requests/CategoryRequest.php

```
/** Used by store (POST) and update (PUT): fields are required only on create. */
class CategoryRequest extends FormRequest
{
    public function rules(): array
    {
        $required = $this->isMethod('post') ? 'required' : 'sometimes';

        return [
            'name' => [$required, 'string', 'max:255'],
            'description' => ['nullable', 'string', 'max:2000'],
            'is_active' => ['boolean'],
        ];
    }
}
```

app/Http/Controllers/Api/CourseCategoryController.php

```
class CourseCategoryController extends Controller
{
    public function index(Request $request): AnonymousResourceCollection
    {
        $isAdmin = (bool) $request->user()?->isAdmin();

        $categories = CourseCategory::query()
            ->unless($isAdmin, fn ($q) => $q->active())
            ->withCount(['courses' => fn ($q) => $q->unless($isAdmin, fn ($q) => $q->published())])
            ->orderBy('name')
            ->get();

        return CategoryResource::collection($categories);
    }

    public function show(Request $request, CourseCategory $category): CategoryResource
    {
        Gate::authorize('view', $category);

        $category->load(['courses' => fn ($q) => $q
            ->unless($request->user()?->isAdmin(), fn ($q) => $q->published())
            ->with(['category', 'instructor'])
            ->withActiveCount()]);

        return new CategoryResource($category);
    }

    public function store(CategoryRequest $request): JsonResponse
    {
        $category = CourseCategory::create($request->validated());

        return (new CategoryResource($category))->response()->setStatusCode(201);
    }

    public function update(CategoryRequest $request, CourseCategory $category): CategoryResource
    {
        $category->update($request->validated());

        return new CategoryResource($category);
    }
```

```
    public function destroy(CourseCategory $category): JsonResponse
    {
        if ($category->courses()->exists()) {
            throw BusinessRuleException::conflict('Move or delete the courses in this category first.');
        }

        $category->delete();

        return response()->json(['message' => 'Category deleted.']);
    }
}
```

VS. THE REPO

The repo's `GET /categories/{id}` was public and returned **every** course in the category, drafts included, and even showed inactive categories. Deleting a category also **cascade-deleted all its courses and their enrollments**.

STEP 15 Courses module

`GET /courses` and `GET /courses/{id}` are public with an optional token: admins also get drafts, and a logged-in student gets their own `enrollment`.

app/Http/Requests/CourseRequest.php

```
class CourseRequest extends FormRequest
{
    public function rules(): array
    {
        $required = $this->isMethod('post') ? 'required' : 'sometimes';

        return [
            'title' => [$required, 'string', 'max:255'],
            'description' => ['nullable', 'string', 'max:5000'],
            'category_id' => [$required, 'integer', 'exists:course_categories,id'],
            'instructor_id' => [$required, 'integer', UserWithRole::of(RoleName::Admin)],
            'level' => [$required, Rule::enum(CourseLevel::class)],
            'price' => [$required, 'numeric', 'min:0', 'max:999999.99'],
            'duration_hours' => ['nullable', 'integer', 'min:1'],
            'image' => ['nullable', 'url:https', 'max:2048'],
            'is_published' => ['boolean'],
            'max_students' => ['nullable', 'integer', 'min:1'],
        ];
    }
}
```

app/Http/Controllers/Api/CourseController.php

```
class CourseController extends Controller
{
    public function index(Request $request): AnonymousResourceCollection
    {
        $filters = $request->validate([
            'category_id' => ['nullable', 'integer'],
            'level' => ['nullable', Rule::enum(CourseLevel::class)],
            'search' => ['nullable', 'string', 'max:100'],
        ]);

        $courses = Course::query()
            ->with(['category', 'instructor'])
            ->withActiveCount()
            ->unless($request->user()?->isAdmin(), fn ($q) => $q->published())
```

```
            ->filter($filters)
            ->latest()
            ->get();

        return CourseResource::collection($courses);
    }

    public function show(Request $request, Course $course): CourseResource
    {
        Gate::authorize('view', $course);

        $course->load(['category', 'instructor'])->loadCount([
            'enrollments' => fn ($q) => $q->where('status', EnrollmentStatus::Active),
        ]);

        // A logged-in student also gets their own enrollment on this course.
        if ($request->user()?->isStudent()) {
            $course->setRelation('enrollment', $course->enrollments()
                ->where('student_id', $request->user()->id)
                ->where('status', '!=', EnrollmentStatus::Dropped)
                ->first());
        }

        return new CourseResource($course);
    }

    public function store(CourseRequest $request): JsonResponse
    {
        $course = Course::create($request->validated());

        return (new CourseResource($course->load(['category', 'instructor'])))
            ->response()->setStatusCode(201);
    }

    public function update(CourseRequest $request, Course $course): CourseResource
    {
        $course->update($request->validated());

        return new CourseResource($course->load(['category', 'instructor']));
    }

    public function destroy(Course $course): JsonResponse
    {
        $course->delete(); // enrollments cascade

        return response()->json(['message' => 'Course deleted.']);
    }
}
```

TIP

Query filters are validated too. `?level=expert` returns a 422 instead of being silently ignored.

VS. THE REPO

`image` must be an `https` URL. The repo accepted any string, from `javascript:` to plain-http URLs, which trigger mixed-content warnings.

STEP 16 Enrollment service: seats, re-enroll, consistent status

The enrollment rules live in one service. The controller stays thin, and the rules can be tested without HTTP.

app/Services/EnrollmentService.php

```
class EnrollmentService
{
    /** Enroll (or re-activate a dropped enrollment). Runs in one transaction so two requests can't both take the
last seat. */
    public function enroll(User $student, Course $course): Enrollment
    {
        return DB::transaction(function () use ($student, $course) {
            $course = Course::lockForUpdate()->findOrFail($course->id);

            if (! $course->is_published) {
                throw new BusinessRuleException('This course is not available for enrollment.');
            }

            $enrollment = Enrollment::firstOrNew([
                'student_id' => $student->id,
                'course_id' => $course->id,
            ]);

            if ($enrollment->exists && $enrollment->status !== EnrollmentStatus::Dropped) {
                throw new BusinessRuleException('Already enrolled in this course.');
            }

            if ($course->isFull()) {
                throw new BusinessRuleException('This course has reached its maximum capacity.');
            }

            $enrollment->fill([
                'status' => EnrollmentStatus::Active,
                'progress' => 0,
                'grade' => null,
                'enrolled_at' => now(),
                'completed_at' => null,
            ])->save();

            return $enrollment;
        });
    }

    /** Admin edits; keeps status, progress and completed_at consistent. */
    public function update(Enrollment $enrollment, array $data): Enrollment
    {
        $enrollment->fill($data);

        if ($enrollment->progress === 100) {
            $enrollment->status = EnrollmentStatus::Completed;
        }

        if ($enrollment->status === EnrollmentStatus::Completed) {
            $enrollment->progress = 100;
            $enrollment->completed_at ??= now();
        } else {
            $enrollment->completed_at = null;
        }

        $enrollment->save();

        return $enrollment;
    }
```

```
    public function drop(Enrollment $enrollment): void
    {
        $enrollment->update(['status' => EnrollmentStatus::Dropped, 'completed_at' => null]);
    }
}
```

Admin sends	Result
progress = 100	status → completed, completed_at = now
status = completed	progress → 100, completed_at kept or set
status = active / dropped	completed_at → null
drop then enroll again	same row re-activated: progress 0, grade null

WHY

The capacity check and the insert run in one transaction. SQLite serialises writers, and on MySQL or Postgres `lockForUpdate()` locks the course row, so two students can't both take the last seat.

VS. THE REPO

The repo compared `$validated['progress'] === 100`. A form-encoded `"100"` never matched, so the enrollment stayed *active* at 100 %. Setting status *completed* by hand left `completed_at` empty.

STEP 17 Enrollments controller

Scoping by role happens in one `match`; every per-record action goes through the policy.

app/Http/Controllers/Api/EnrollmentController.php

```
class EnrollmentController extends Controller
{
    private const WITH = ['student', 'course.category'];

    public function __construct(private EnrollmentService $enrollments) {}

    /** Admin: all. Student: own. Parent: their children's. */
    public function index(Request $request): AnonymousResourceCollection
    {
        $user = $request->user();

        $query = match (true) {
            $user->isAdmin() => Enrollment::query(),
            $user->isStudent() => Enrollment::where('student_id', $user->id),
            $user->isParent() => Enrollment::whereIn('student_id', $user->children()->select('id')),
            default => abort(403),
        };

        return EnrollmentResource::collection($query->with(self::WITH)->latest()->get());
    }

    public function store(StoreEnrollmentRequest $request): JsonResponse
    {
        Gate::authorize('create', Enrollment::class);

        $enrollment = $this->enrollments->enroll(
            User::findOrFail($request->studentId()),
            Course::findOrFail($request->validated('course_id')),
        );

        return (new EnrollmentResource($enrollment->load(self::WITH)))
```

```
            ->response()->setStatusCode(201);
    }

    public function show(Enrollment $enrollment): EnrollmentResource
    {
        Gate::authorize('view', $enrollment);

        return new EnrollmentResource($enrollment->load(self::WITH));
    }

    public function update(UpdateEnrollmentRequest $request, Enrollment $enrollment): EnrollmentResource
    {
        Gate::authorize('update', $enrollment);

        $enrollment = $this->enrollments->update($enrollment, $request->validated());

        return new EnrollmentResource($enrollment->load(self::WITH));
    }

    public function destroy(Enrollment $enrollment): JsonResponse
    {
        Gate::authorize('delete', $enrollment);

        $this->enrollments->drop($enrollment);

        return response()->json(['message' => 'Enrollment dropped.']);
    }
}
```

> **TIP**
> `$user->children()->select('id')` becomes a sub-query (`WHERE student_id IN (SELECT id …)`): one query instead of loading the IDs first.

STEP 18 Users module (admin)

Admins list, create and delete users. Anyone may view and update their own account, and parents may view their children.

app/Http/Controllers/Api/UserController.php

```
class UserController extends Controller
{
    /** Admin only (route middleware). ?role=student&search=jane */
    public function index(Request $request): AnonymousResourceCollection
    {
        $filters = $request->validate([
            'role' => ['nullable', Rule::enum(RoleName::class)],
            'search' => ['nullable', 'string', 'max:100'],
        ]);

        $users = User::query()
            ->when($filters['role'] ?? null, fn ($q, $role) => $q->withRole(RoleName::from($role)))
            ->when($filters['search'] ?? null, fn ($q, $term) => $q->where(fn ($q) => $q
                ->where('name', 'like', "%{$term}%")
                ->orWhere('email', 'like', "%{$term}%")))
            ->orderBy('name')
            ->get();

        return UserResource::collection($users);
    }

    public function store(UserRequest $request): JsonResponse
```

```
    {
        $user = User::create($request->validated());

        return (new UserResource($user->load('role')))->response()->setStatusCode(201);
    }

    public function show(User $user): UserResource
    {
        Gate::authorize('view', $user);

        return new UserResource($user->load(['parent', 'children', 'enrollments.course.category']));
    }

    public function update(UserRequest $request, User $user): JsonResponse|UserResource
    {
        Gate::authorize('update', $user);
        $data = $request->validated();

        if (isset($data['role_id']) && (int) $data['role_id'] !== $user->role_id) {
            Gate::authorize('changeRole', $user);
        }

        $user->update($data);

        return new UserResource($user->load('role'));
    }

    public function destroy(User $user): JsonResponse
    {
        Gate::authorize('delete', $user);

        if ($user->taughtCourses()->exists()) {
            throw BusinessRuleException::conflict('Reassign this instructor\'s courses before deleting the
account.');
        }

        $user->delete(); // enrollments cascade, children are unlinked (parent_id -> null)

        return response()->json(['message' => 'User deleted.']);
    }
}
```

VS. THE REPO

In the repo, deleting an admin **deleted every course they taught, and every enrollment in those courses** (`instructor_id` was `cascadeOnDelete`). The API now refuses with a 409, and the foreign key is `restrict` as a safety net.

STEP 19 Dashboard: one endpoint, three shapes

A single-action controller (`__invoke`). The response keys match what the React dashboards already read.

app/Http/Controllers/Api/DashboardController.php

```
class DashboardController extends Controller
{
    public function __invoke(Request $request): JsonResponse
    {
        $user = $request->user();

        return response()->json(match ($user->role?->name) {
            RoleName::Admin => $this->adminStats(),
            RoleName::Student => $this->studentStats($user),
```

```
            RoleName::Parent => $this->parentStats($user),
            default => abort(403, 'No dashboard available.'),
        });
    }

    private function adminStats(): array
    {
        $byStatus = Enrollment::toBase()->selectRaw('status, count(*) as total')
            ->groupBy('status')->pluck('total', 'status');

        return [
            'total_students' => User::withRole(RoleName::Student)->count(),
            'total_parents' => User::withRole(RoleName::Parent)->count(),
            'total_courses' => Course::count(),
            'published_courses' => Course::published()->count(),
            'total_categories' => CourseCategory::count(),
            'total_enrollments' => $byStatus->sum(),
            'active_enrollments' => (int) $byStatus->get(EnrollmentStatus::Active->value, 0),
            'completed_enrollments' => (int) $byStatus->get(EnrollmentStatus::Completed->value, 0),
            'recent_enrollments' => EnrollmentResource::collection(
                Enrollment::with(['student', 'course'])->latest()->take(5)->get()
            ),
            'popular_courses' => CourseResource::collection(
                Course::withActiveCount()->orderByDesc('enrollments_count')->take(5)->get()
            ),
        ];
    }

    private function studentStats(User $student): array
    {
        $enrollments = $student->enrollments()->with('course.category')->latest()->get();

        return [...$this->summarize($enrollments), 'enrollments' => EnrollmentResource::collection($enrollments)];
    }

    private function parentStats(User $parent): array
    {
        $children = $parent->children()->get();
        $enrollments = Enrollment::whereIn('student_id', $children->modelKeys())
            ->with(['student', 'course.category'])->latest()->get();

        return [
            'children' => $children->map(fn (User $child) => [
                'student' => new UserResource($child),
                ...$this->summarize($enrollments->where('student_id', $child->id)),
            ]),
            'enrollments' => EnrollmentResource::collection($enrollments),
        ];
    }

    /** Same numbers for a student's own dashboard and for each child of a parent. */
    private function summarize(Collection $enrollments): array
    {
        $active = $enrollments->where('status', EnrollmentStatus::Active);

        return [
            'total_enrolled' => $enrollments->count(),
            'completed' => $enrollments->where('status', EnrollmentStatus::Completed)->count(),
            'in_progress' => $active->count(),
            'average_progress' => round($active->avg('progress') ?? 0, 1),
            'average_grade' => round($enrollments->whereNotNull('grade')->avg('grade') ?? 0, 1),
        ];
    }
}
```

> **TIP**
> Enrollment counts by status come from one `GROUP BY` query instead of three `COUNT`s. Student and per-child stats share `summarize()`, so the numbers can't drift apart.

STEP 20 Routes

`apiResource` keeps the routes short. `{category}` replaces the repo's long `{courseCategory}` parameter name.

routes/api.php

```
// ── Public ──────────────────────────────────────────────────────
Route::prefix('auth')->group(function () {
    Route::middleware('throttle:auth')->group(function () {
        Route::post('register', [AuthController::class, 'register']);
        Route::post('login', [AuthController::class, 'login']);
    });
    Route::get('roles', [AuthController::class, 'roles']);
    Route::get('parents', [AuthController::class, 'parents']);
});

// A bearer token is optional here: admins also see drafts, students get their enrollment.
Route::apiResource('courses', CourseController::class)->only(['index', 'show']);
Route::apiResource('categories', CourseCategoryController::class)
    ->only(['index', 'show'])->parameters(['categories' => 'category']);

// ── Authenticated ───────────────────────────────────────────────
Route::middleware('auth:sanctum')->group(function () {
    Route::prefix('auth')->group(function () {
        Route::post('logout', [AuthController::class, 'logout']);
        Route::get('me', [AuthController::class, 'me']);
        Route::put('profile', [AuthController::class, 'updateProfile']);
    });

    Route::get('dashboard', DashboardController::class);

    // Record-level rules live in the policies.
    Route::apiResource('enrollments', EnrollmentController::class);
    Route::apiResource('users', UserController::class)->only(['show', 'update']);

    Route::middleware('role:admin')->group(function () {
        Route::apiResource('users', UserController::class)->only(['index', 'store', 'destroy']);
        Route::apiResource('courses', CourseController::class)->only(['store', 'update', 'destroy']);
        Route::apiResource('categories', CourseCategoryController::class)
            ->only(['store', 'update', 'destroy'])->parameters(['categories' => 'category']);
    });
});
```

Method	Path (prefix /api)	Access
POST	/auth/register · /auth/login	public, throttle:auth
GET	/auth/roles · /auth/parents	public
GET	/courses · /courses/{id}	public (token optional)
GET	/categories · /categories/{id}	public (token optional)
POST/GET/PUT	/auth/logout · /auth/me · /auth/profile	token
GET	/dashboard	token
GET POST GET PUT DELETE	/enrollments[/{id}]	token + EnrollmentPolicy

Method	Path (prefix /api)	Access
GET PUT	/users/{id}	token + UserPolicy
GET POST DELETE	/users[/{id}]	role:admin
POST PUT DELETE	/courses[/{id}] · /categories[/{id}]	role:admin

STEP 21 Try it with curl

These calls ran against the seeded database (`migrate:fresh --seed` + `php artisan serve`). The outputs are real.

terminal

```
B=http://127.0.0.1:8000/api
H='-H Accept:application/json -H Content-Type:application/json'

curl $H -X POST $B/auth/login -d '{"email":"student@school.com","password":"password"}'
# {"user":{"id":3,"name":"Student User","email":"student@school.com","role_id":2,
#  "parent_id":2,...,"role":{"id":2,"name":"student"}},"token":"1|SYfH…"}
T=1|SYfH…

curl $H $B/courses/1                      # public: instructor is {id, name} only
# {"id":1,"title":"Algebra Fundamentals",...,"enrollments_count":1,
#  "instructor":{"id":1,"name":"Admin User"}}

curl $H -H "Authorization: Bearer $T" -X POST $B/enrollments -d '{"course_id":3}'   # 201
curl $H -H "Authorization: Bearer $T" -X POST $B/enrollments -d '{"course_id":3}'
# {"message":"Already enrolled in this course."}            422

curl $H -H "Authorization: Bearer $T" -X PUT $B/enrollments/1 -d '{"grade":100}'
# {"message":"This action is unauthorized."}                403

curl $H $B/dashboard
# {"message":"Unauthenticated. Please log in again."}       401

curl $H -H "Authorization: Bearer $T" -X POST $B/auth/logout                          # 200
curl $H -H "Authorization: Bearer $T" $B/auth/me                                      # 401

for i in 1 2 3 4 5 6; do curl -s -o /dev/null -w '%{http_code} ' $H \
  -X POST $B/auth/login -d '{"email":"x@x.com","password":"bad"}'; done
# 422 422 422 422 422 429

curl $H -H "Authorization: Bearer $ADMIN" -X DELETE $B/categories/1
# {"message":"Move or delete the courses in this category first."}   409
```

WATCH OUT

Always send `Accept: application/json`. Without it, a non-API path can answer with an HTML redirect instead of a JSON error.

STEP 22 Backend lifecycle cheat table

One request from start to finish, with the file that owns each stage.

#	Stage	Owner	Fails with
1	CORS preflight / headers	`config/cors.php`	—
2	Default guard → sanctum	`UseSanctumGuard`	—
3	Rate limit	`throttle:api`, `throttle:auth`	429
4	Token check	`auth:sanctum`	401
5	Role gate	`EnsureRole` (`role:admin`)	403
6	Route-model binding	`{course}` → `Course`	404
7	Validation	FormRequest	422
8	Record-level permission	Policy via `Gate::authorize`	403 / 404
9	Business rules	`EnrollmentService`, controller checks	422 / 409
10	Persist	Eloquent + FK constraints	—
11	Serialize	API Resource (no wrapping)	200 / 201
12	Errors anywhere	`bootstrap/app.php` → JSON `{message}`	—

PART C

Database

Schema, relations, delete rules, indexes and seed data.

STEP 23 Schema & relations

There are five domain tables. Users relate to themselves (parent → children) and to courses twice: as instructor and, through enrollments, as student.

```
roles 1 ─────< users >──── 0..1 users (parent_id → parent)
                 │  │
  instructor_id  │  │ student_id
                 ▼  ▼
course_categories 1 ──< courses 1 ──< enrollments >── 1 users
                                      unique(student_id, course_id)
```

Table	Key columns	Relations
roles	name unique (admin/student/parent)	`hasMany users`
users	role_id, parent_id, phone, address, date_of_birth	role, parent, children, enrollments, taughtCourses
course_categories	name, slug unique, is_active	`hasMany courses`
courses	slug unique, level, price dec(8,2), is_published, max_students	category, instructor, enrollments
enrollments	status, progress 0–100, grade dec(5,2), enrolled_at, completed_at	student, course
personal_access_tokens	token (sha256) unique, expires_at	Sanctum: morphs tokenable

STEP 24 Migrations: delete rules & indexes

Delete rules decide what one `DELETE` can destroy. Every foreign key states its rule explicitly.

database/migrations/2026_10_12_141741_create_courses_table.php

```php
    Schema::create('courses', function (Blueprint $table) {
        $table->id();
        $table->string('title');
        $table->string('slug')->unique();
        $table->text('description')->nullable();
        // restrict: deleting a category or an instructor must never wipe courses
        $table->foreignId('category_id')->constrained('course_categories')->restrictOnDelete();
        $table->foreignId('instructor_id')->constrained('users')->restrictOnDelete();
        $table->string('level')->default('beginner');
        $table->decimal('price', 8, 2)->default(0);
        $table->unsignedInteger('duration_hours')->nullable();
        $table->string('image')->nullable();
        $table->boolean('is_published')->default(false);
        $table->unsignedInteger('max_students')->nullable();
        $table->timestamps();

        $table->index(['is_published', 'category_id']);
```

```
    });
}
```

database/migrations/2026_10_12_141742_create_enrollments_table.php

```
    Schema::create('enrollments', function (Blueprint $table) {
        $table->id();
        $table->foreignId('student_id')->constrained('users')->cascadeOnDelete();
        $table->foreignId('course_id')->constrained('courses')->cascadeOnDelete();
        $table->string('status')->default('active');
        $table->unsignedTinyInteger('progress')->default(0);
        $table->decimal('grade', 5, 2)->nullable();
        $table->timestamp('enrolled_at')->useCurrent();
        $table->timestamp('completed_at')->nullable();
        $table->timestamps();

        $table->unique(['student_id', 'course_id']);
        $table->index(['course_id', 'status']); // capacity checks + popular courses
    });
}
```

database/migrations/2026_10_12_141743_add_role_to_users_table.php

```
    Schema::table('users', function (Blueprint $table) {
        $table->foreignId('role_id')->nullable()->constrained('roles')->restrictOnDelete();
        $table->foreignId('parent_id')->nullable()->constrained('users')->nullOnDelete();
        $table->string('phone', 20)->nullable();
        $table->text('address')->nullable();
        $table->date('date_of_birth')->nullable();
    });
}
```

Foreign key	Repo	Now	Why
courses.category_id	cascade	**restrict**	deleting a category wiped its courses
courses.instructor_id	cascade	**restrict**	deleting an admin wiped their courses
users.role_id	set null	**restrict**	a user without a role passes no check and has no dashboard
users.parent_id	set null	set null	child stays, link removed
enrollments.*	cascade	cascade	history belongs to the student and course

WATCH OUT

These are edits to migrations that already ran. On a database you need to keep, add a **new** migration that drops and re-creates the foreign keys. On SQLite, Laravel rebuilds the table for you. For local dev, `php artisan migrate:fresh --seed` is enough.

TIP

New indexes: `courses(is_published, category_id)` for the catalog filter and `enrollments(course_id, status)` for capacity checks and "popular courses".

STEP 25 Seeding & demo accounts

The seeder builds roles, four demo users (password `password`), six categories, nine published courses and three enrollments.

database/seeders/DatabaseSeeder.php

```
public function run(): void
{
    $roles = collect([
        RoleName::Admin->value => 'System administrator',
        RoleName::Student->value => 'Enrolled student',
        RoleName::Parent->value => 'Parent / guardian',
    ])->map(fn ($description, $name) => Role::create(compact('name', 'description')));

    // Demo accounts, all with password "password" (hashed by the model cast).
    $admin = User::create(['name' => 'Admin User', 'email' => 'admin@school.com',
        'password' => 'password', 'role_id' => $roles['admin']->id]);
    $parent = User::create(['name' => 'Parent User', 'email' => 'parent@school.com',
        'password' => 'password', 'role_id' => $roles['parent']->id, 'phone' => '+1-555-0100']);
    $student = User::create(['name' => 'Student User', 'email' => 'student@school.com',
        'password' => 'password', 'role_id' => $roles['student']->id,
        'parent_id' => $parent->id, 'date_of_birth' => '2006-05-15']);
    $jane = User::create(['name' => 'Jane Smith', 'email' => 'jane@school.com',
        'password' => 'password', 'role_id' => $roles['student']->id,
        'parent_id' => $parent->id, 'date_of_birth' => '2007-09-20']);
```

Email	Role	Data
admin@school.com	admin	instructor of all courses
parent@school.com	parent	parent of Student User and Jane
student@school.com	student	1 active (65 %), 1 completed (92.5)
jane@school.com	student	1 active (30 %)

WATCH OUT

Never run this seeder in production. Guard it with `if (app()->isProduction()) return;` or use a separate `ProductionSeeder` that only creates roles.

PART D

Security

What can go wrong, what now prevents it, and what to do before going live.

STEP 26 Threat model & OWASP mapping

Each row is a real way to attack this API, with the control that stops it and where that control lives.

Threat	OWASP 2021	Control	See
Password brute force	A07 Auth failures	`throttle:auth` 5/min per email + IP	Step 11
Stolen token → account takeover	A07	`current_password` + revoke other tokens; 7-day expiry	Step 12
Self-register as admin	A01 Broken access control	`Rule::enum()->only(student, parent)`	Step 09
Student grades themselves	A01	`EnrollmentPolicy::update` = admin only	Step 13
Parent reads a stranger's data	A01	`isParentOf()` in policies	Step 13
Self-promotion via `PUT /users/{me}`	A01	admin-only fields dropped in `UserRequest`	Step 09
PII leak in the public catalog	A01 / A04 Insecure design	Resources expose instructor `{id, name}` only	Step 10
Draft/inactive enumeration	A01	`denyAsNotFound()` → 404	Step 13
Mass deletion via cascades	A04	`restrict` FKs + 409 checks	Step 24
SQL injection	A03 Injection	Eloquent bindings everywhere, no raw input in SQL	—
Mass assignment	A08 Integrity	`#[Fillable]` + validated() only + strict mode	Step 08
Leaky errors	A05 Misconfiguration	`APP_DEBUG=false` in prod; JSON `{message}`	Step 27
Abuse / scraping	A04	`throttle:api` 120/min per user or IP	Step 08

WATCH OUT

`GET /auth/parents` is public and lists every parent's name. Any student can link themselves to any parent when registering, and that parent then sees the student's grades. This is kept for compatibility with the current register form. A safer design: the parent invites a child with a one-time code, or an admin sets the link.

STEP 27 Production hardening checklist

Run through this list before the first deploy.

	Item	Why
☐	`APP_ENV=production`, `APP_DEBUG=false`, fresh `APP_KEY`	Debug pages print file paths and stack traces (as in the 403 from Step 21)
☐	`CORS_ALLOWED_ORIGINS` = the real SPA origin only	Stops other sites from reading API responses
☐	HTTPS only; HSTS at the proxy	Bearer tokens are passwords in transit
☐	`php artisan config:cache route:cache`	Speed. `env()` must only be called in `config/`
☐	Cron for `schedule:run`	Prunes expired tokens nightly
☐	Seeder demo accounts not created	`password` is the first thing an attacker tries
☐	`composer audit` in CI	Known-vulnerable dependencies
☐	`database/database.sqlite` outside the web root, backed up	Only `public/` should be served
☐	Log 401/403/429 spikes (`LOG_LEVEL=warning`)	Early sign of brute force or scraping

deploy (excerpt)

```
composer install --no-dev --optimize-autoloader
php artisan migrate --force
php artisan config:cache && php artisan route:cache
composer audit
```

PART E

Testing

Feature tests over HTTP against in-memory SQLite: 25 tests, 77 assertions.

STEP 28 Test setup & helpers

`phpunit.xml` already points the tests at an in-memory SQLite database, so every test class with `RefreshDatabase` starts from a clean schema.

phpunit.xml (excerpt)

```
<env name="APP_ENV" value="testing"/>
<env name="BCRYPT_ROUNDS" value="4"/>     <!-- fast hashing in tests -->
<env name="CACHE_STORE" value="array"/>    <!-- rate limiter resets per test -->
<env name="DB_CONNECTION" value="sqlite"/>
<env name="DB_DATABASE" value=":memory:"/>
```

tests/TestCase.php

```
abstract class TestCase extends BaseTestCase
{
    protected function makeUser(RoleName $role, array $attributes = []): User
    {
        $roleId = Role::firstOrCreate(['name' => $role])->id;

        return User::create([
            'name' => ucfirst($role->value),
            'email' => $role->value.uniqid().'@test.com',
            'password' => 'password',
            'role_id' => $roleId,
            ...$attributes,
        ]);
    }

    protected function makeCourse(?User $instructor = null, array $attributes = []): Course
    {
        $category = CourseCategory::firstOrCreate(['name' => 'Cat'], ['is_active' => true]);

        return Course::create([
            'title' => 'Course',
            'category_id' => $category->id,
            'instructor_id' => ($instructor ?? $this->makeUser(RoleName::Admin))->id,
            'level' => 'beginner',
            'price' => 0,
            'is_published' => true,
            ...$attributes,
        ]);
    }
}
```

Helper	Use
`Sanctum::actingAs($user)`	skip login; request is authenticated as $user
`->withToken($plain)`	real token, needed to test logout and revocation
`assertJsonValidationErrors('field')`	422 with that key in `errors`

Helper	Use
`assertConflict()` / `assertTooManyRequests()`	409 / 429

STEP 29 Feature tests: auth, catalog, enrollments

Each fix from this guide has a test that fails on the old code. Three files group them by area.

tests/Feature/AuthTest.php

```
public function test_login_is_throttled_after_five_attempts(): void
{
    foreach (range(1, 5) as $_) {
        $this->postJson('/api/auth/login', ['email' => 'a@b.com', 'password' => 'nope'])->assertUnprocessable();
    }

    $this->postJson('/api/auth/login', ['email' => 'a@b.com', 'password' => 'nope'])->assertTooManyRequests();
}

public function test_password_change_needs_current_password_and_revokes_other_tokens(): void
{
    $user = $this->makeUser(RoleName::Student);
    $current = $user->createToken('this-device')->plainTextToken;
    $user->createToken('other-device');

    $this->withToken($current)->putJson('/api/auth/profile', [
        'password' => 'newpassword', 'password_confirmation' => 'newpassword',
    ])->assertJsonValidationErrors('current_password');

    $this->withToken($current)->putJson('/api/auth/profile', [
        'password' => 'newpassword', 'password_confirmation' => 'newpassword', 'current_password' => 'password',
    ])->assertOk();

    $this->assertSame(1, $user->tokens()->count());
}
```

tests/Feature/CatalogTest.php

```
public function test_public_catalog_hides_drafts_and_instructor_contact_details(): void
{
    $instructor = $this->makeUser(RoleName::Admin, ['phone' => '+1-555-0000']);
    $this->makeCourse($instructor, ['title' => 'Live']);
    $this->makeCourse($instructor, ['title' => 'Draft', 'is_published' => false]);

    $this->getJson('/api/courses')->assertOk()
        ->assertJsonCount(1)
        ->assertJsonPath('0.instructor', ['id' => $instructor->id, 'name' => $instructor->name]);
}
```

tests/Feature/EnrollmentTest.php

```
public function test_parent_sees_only_their_children_and_cannot_drop(): void
{
    $course = $this->makeCourse();
    $parent = $this->makeUser(RoleName::Parent);
    $child = $this->makeUser(RoleName::Student, ['parent_id' => $parent->id]);
    $stranger = $this->makeUser(RoleName::Student);

    Sanctum::actingAs($child);
    $mine = $this->postJson('/api/enrollments', ['course_id' => $course->id])->json('id');
    Sanctum::actingAs($stranger);
    $theirs = $this->postJson('/api/enrollments', ['course_id' => $course->id])->json('id');
```

```
    Sanctum::actingAs($parent);
    $this->getJson('/api/enrollments')->assertOk()->assertJsonCount(1)->assertJsonPath('0.id', $mine);
    $this->getJson("/api/enrollments/$theirs")->assertForbidden();
    $this->deleteJson("/api/enrollments/$mine")->assertForbidden();
}
```

File	Covers
AuthTest	register role whitelist · login + token · parent_id must be a parent · throttle · password change revokes · logout revokes · 401 · 403
CatalogTest	unique slugs · public PII hidden · draft 404 · category hides drafts · 409 category · 409 instructor · admin self-protect
EnrollmentTest	drop + re-enroll · capacity · admin enrolls students only · no self-grading · 100 % completes · parent scope · course shows enrollment · dashboards

terminal

```
php artisan test                      # 25 passed (77 assertions)
php artisan test --filter=Enrollment  # one file / one test
vendor/bin/pint --test                # style check (CI)
```

PART F

Reference

Environment, commands, version notes and the full list of fixes.

STEP 30 Environment variables & commands

These are the variables this API reads; everything else is the stock Laravel `.env`.

Variable	Default	Purpose
`APP_KEY`	—	encryption key; `php artisan key:generate`
`APP_DEBUG`	true	**false** in production
`APP_URL`	http://localhost:8000	absolute URLs
`DB_CONNECTION`	sqlite	switch to mysql/pgsql + DB_HOST… later
`CORS_ALLOWED_ORIGINS`	localhost:3000, 127.0.0.1:3000	comma-separated SPA origins
`SANCTUM_EXPIRATION`	10080	token lifetime, minutes
`BCRYPT_ROUNDS`	12	password hashing cost

Command	Does
`php artisan serve`	dev server on :8000
`php artisan migrate:fresh --seed`	rebuild DB with demo data
`php artisan route:list --path=api`	every endpoint + middleware
`php artisan test`	run the suite
`php artisan tinker`	REPL: `User::first()->tokens`
`php artisan sanctum:prune-expired --hours=24`	delete expired tokens now
`php artisan schedule:work`	run the scheduler locally
`vendor/bin/pint`	fix code style
`php artisan pail`	tail logs live

STEP 31 Version notes & upgrade path

If you're coming from Laravel 10 tutorials, these are the differences you'll hit.

Thing	Status	Now
App/Http/Kernel.php	gone (11+)	middleware in `bootstrap/app.php` → `withMiddleware()`
Exception Handler class	gone (11+)	`withExceptions()` in `bootstrap/app.php`
routes/api.php	opt-in (11+)	`php artisan install:api` adds it + Sanctum
AuthServiceProvider / policy map	gone (11+)	policies auto-discovered by name
`$this->authorize()` in controllers	base Controller is empty (11+)	`Gate::authorize()` or the `AuthorizesRequests` trait
`protected $casts`	still works	prefer `casts(): array` method (11+)
`scopeX()` methods	still works	`#[Scope]` attribute (12+)
`$fillable` / `$hidden`	still works	`#[Fillable]` / `#[Hidden]` class attributes (13)
PHP	8.2+ (12)	**8.3+** (13); this project runs 8.4

> **NOTE**
> Laravel 13 gets bug fixes until Q3 2027 and security fixes until Q1 2028.

STEP 32 Bugs & gaps fixed vs. the repo

Every change in this guide, with its severity and where the fix lives. Tests cover each high and medium item.

Sev.	Problem in the repo	Fix	Step
High	Deleting an admin cascade-deleted all their courses + enrollments	restrict FK + 409	18, 24
High	Deleting a category cascade-deleted its courses + enrollments	restrict FK + 409	14, 24
High	Public `GET /courses` leaked instructor email, phone, address, DOB	API Resources	10
High	No rate limit on login/register (brute force)	`throttle:auth`	11
Med	Password change without current password; other sessions kept	`current_password` + revoke	12
Med	Public category page listed drafts and inactive categories	policy + scoped load	14
Med	`parent_id` could be any user (admin form)	`UserWithRole`	09
Med	Admin could enroll a parent/admin as a student	`StoreEnrollmentRequest`	09
Med	Enroll capacity check not atomic (last seat race)	transaction + lock	16
Med	Expired tokens never pruned	scheduler	12
Low	`progress === 100` missed string input; completed_at inconsistent	`EnrollmentService::update`	16
Low	Role deletion set users' role to null	restrict FK	24
Low	`image` accepted any string	`url:https`	15
Low	Query filters unvalidated (`level` , `role`)	inline validate	15, 18
Low	Drafts answered 403 (reveals existence)	`denyAsNotFound()`	13

Sev.	Problem in the repo	Fix	Step
Clean	Slug loop copied 3×; magic strings; fat controllers	trait, enums, FormRequests, policies	05–09
Clean	No global API rate limit; no strict model mode	`throttleApi()`, `shouldBeStrict()`	08

> NOTE
> **Frontend impact:** one change. `Profile.jsx` must send `current_password` when changing the password. All response shapes the SPA reads are unchanged.